EXPERIMENT : 3

Aim

To create an Employee–Department–Project database containing exactly 30 employees, 5 departments, and 8 projects, and to execute SQL queries demonstrating selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Theory

SQL provides commands for storing, retrieving, grouping, and analyzing relational data. Selection retrieves records based on a condition, while projection retrieves specific columns. Aggregate functions perform calculations on multiple records. GROUP BY forms groups for aggregate operations, and HAVING filters those groups. CASE provides conditional expressions, while ORDER BY sorts query results.

Source Code

1. Create Database and Department Table

```
CREATE DATABASE employee_management;

USE employee_management;

CREATE TABLE Department (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50) NOT NULL,
   location VARCHAR(50)
);
```

2. Insert 5 Departments

```
INSERT INTO Department VALUES
(1, 'Computer Science', 'Chandigarh'),
(2, 'Human Resources', 'Delhi'),
(3, 'Finance', 'Mumbai'),
(4, 'Marketing', 'Bangalore'),
(5, 'Research', 'Pune');
```

3. Create Employee Table

```
CREATE TABLE Employee (
```

```
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    job_title VARCHAR(50),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

4. Insert Exactly 30 Employees

```
INSERT INTO Employee VALUES
(101, 'Aarav Sharma', 55000, 'Software Engineer', 1),
(102, 'Rohit Verma', 62000, 'Data Analyst', 1),
(103, 'Aditya Singh', 48000, 'Web Developer', 1),
(104, 'Karan Mehta', 70000, 'Senior Developer', 1),
(105, 'Vivek Kumar', 52000, 'System Analyst', 1),
(106, 'Arjun Patel', 65000, 'Database Engineer', 1),

(107, 'Neha Sharma', 45000, 'HR Executive', 2),
(108, 'Priya Singh', 58000, 'HR Manager', 2),
(109, 'Anjali Verma', 42000, 'Recruiter', 2),
(110, 'Sneha Kapoor', 60000, 'HR Analyst', 2),
(111, 'Pooja Mehta', 50000, 'Training Officer', 2),
(112, 'Riya Gupta', 72000, 'HR Manager', 2),

(113, 'Rahul Jain', 68000, 'Financial Analyst', 3),
(114, 'Aman Gupta', 52000, 'Accountant', 3),
(115, 'Nikhil Sharma', 75000, 'Finance Manager', 3),
(116, 'Varun Singh', 47000, 'Accountant', 3),
(117, 'Mohit Kumar', 59000, 'Financial Analyst', 3),
```

(118, 'Yash Patel', 81000, 'Senior Accountant', 3),

(119, 'Simran Kaur', 54000, 'Marketing Executive', 4),

(120, 'Isha Sharma', 61000, 'Marketing Analyst', 4),

(121, 'Tanya Verma', 49000, 'Content Manager', 4),

(122, 'Manish Gupta', 67000, 'Marketing Manager', 4),

(123, 'Akash Jain', 46000, 'SEO Executive', 4),

(124, 'Varun Mehta', 73000, 'Brand Manager', 4),

(125, 'Sahil Kumar', 65000, 'Research Analyst', 5),

(126, 'Dev Sharma', 58000, 'Research Associate', 5),

(127, 'Harsh Singh', 76000, 'Research Scientist', 5),

(128, 'Ayush Verma', 51000, 'Research Associate', 5),

(129, 'Manav Gupta', 69000, 'Data Scientist', 5),

(130, 'Kabir Jain', 85000, 'Research Scientist', 5);

5. Create Project Table

```
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    budget DECIMAL(12,2),
    start_date DATE
);
```

6. Insert Exactly 8 Projects

```
INSERT INTO Project VALUES
(201, 'AI Analytics Platform', 1500000, '2026-01-10'),
(202, 'Employee Management System', 800000, '2026-02-15'),
(203, 'Banking Security System', 1800000, '2026-03-01'),
(204, 'E-Commerce Platform', 1200000, '2026-03-20'),
```

```
(205, 'Marketing Automation', 700000, '2026-04-05'),
(206, 'Data Research Portal', 1100000, '2026-04-18'),
(207, 'Cloud Migration', 2000000, '2026-05-01'),
(208, 'Cybersecurity Dashboard', 1700000, '2026-05-15');
```

7. Create Employee_Project Table

```
CREATE TABLE Employee_Project (
    emp_id INT,
    project_id INT,
    role VARCHAR(50),
    PRIMARY KEY (emp_id, project_id),
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

8. Assign Employees to Projects

```
INSERT INTO Employee_Project VALUES
(101, 201, 'Developer'),
(102, 201, 'Data Analyst'),
(103, 202, 'Developer'),
(104, 207, 'Team Lead'),
(105, 208, 'System Analyst'),
(106, 203, 'Database Engineer'),
(107, 202, 'HR Coordinator'),
(108, 202, 'HR Manager'),
(109, 205, 'Recruiter'),
(110, 205, 'HR Analyst'),
(111, 206, 'Training Coordinator'),
(112, 207, 'HR Manager'),
(113, 203, 'Financial Analyst'),
```

(114, 203, 'Accountant'),

(115, 203, 'Finance Manager'),

(116, 204, 'Accountant'),

(117, 206, 'Financial Analyst'),

(118, 207, 'Senior Accountant'),

(119, 205, 'Marketing Executive'),

(120, 205, 'Marketing Analyst'),

(121, 204, 'Content Manager'),

(122, 204, 'Marketing Manager'),

(123, 205, 'SEO Executive'),

(124, 207, 'Brand Manager'),

(125, 206, 'Research Analyst'),

(126, 206, 'Research Associate'),

(127, 201, 'Research Scientist'),

(128, 208, 'Research Associate'),

(129, 201, 'Data Scientist'),

(130, 208, 'Research Scientist');

SQL Queries

1. Selection

```
SELECT *
FROM Employee
WHERE salary > 60000;
```

2. Projection

```
SELECT emp_name, job_title, salary
FROM Employee;
```

3. Aggregate Functions

```
SELECT COUNT(*) AS total_employees,
    AVG(salary) AS average_salary,
```

```
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM Employee;
```

4. GROUP BY

```
SELECT dept_id, COUNT(*) AS employee_count
FROM Employee
GROUP BY dept_id;
```

5. HAVING

```
SELECT dept_id, AVG(salary) AS average_salary
FROM Employee
GROUP BY dept_id
HAVING AVG(salary) > 60000;
```

6. CASE Expression

```
SELECT emp_name, salary,
    CASE
        WHEN salary >= 70000 THEN 'High'
        WHEN salary >= 55000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_category
FROM Employee;
```

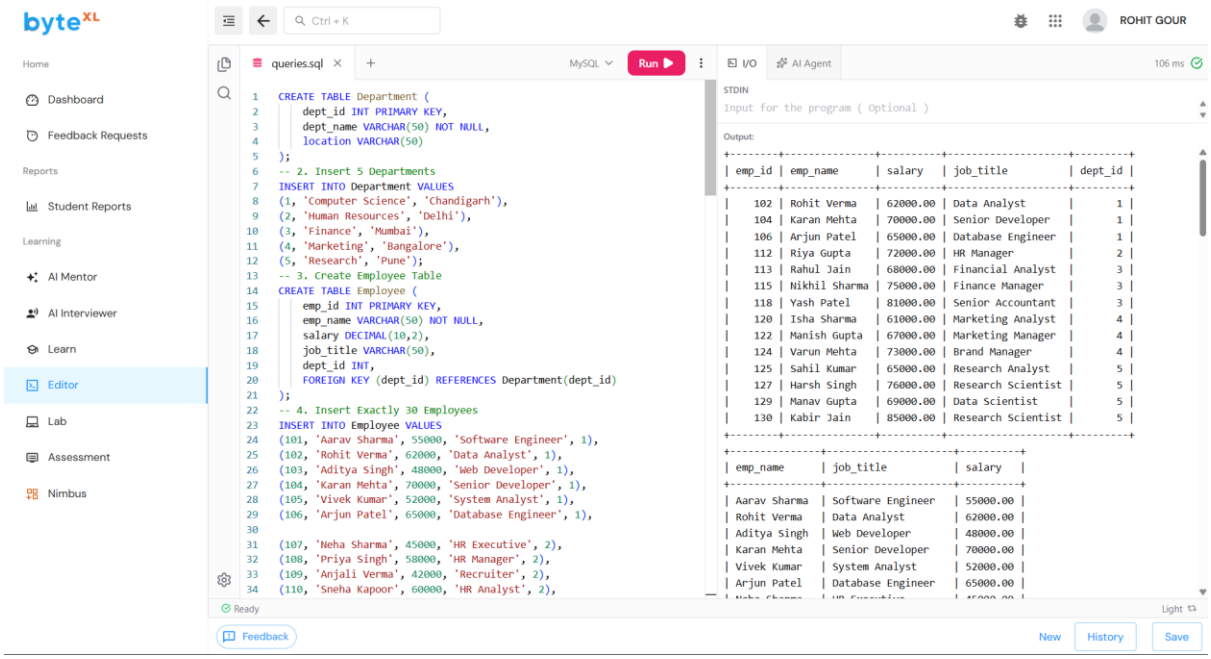
7. ORDER BY

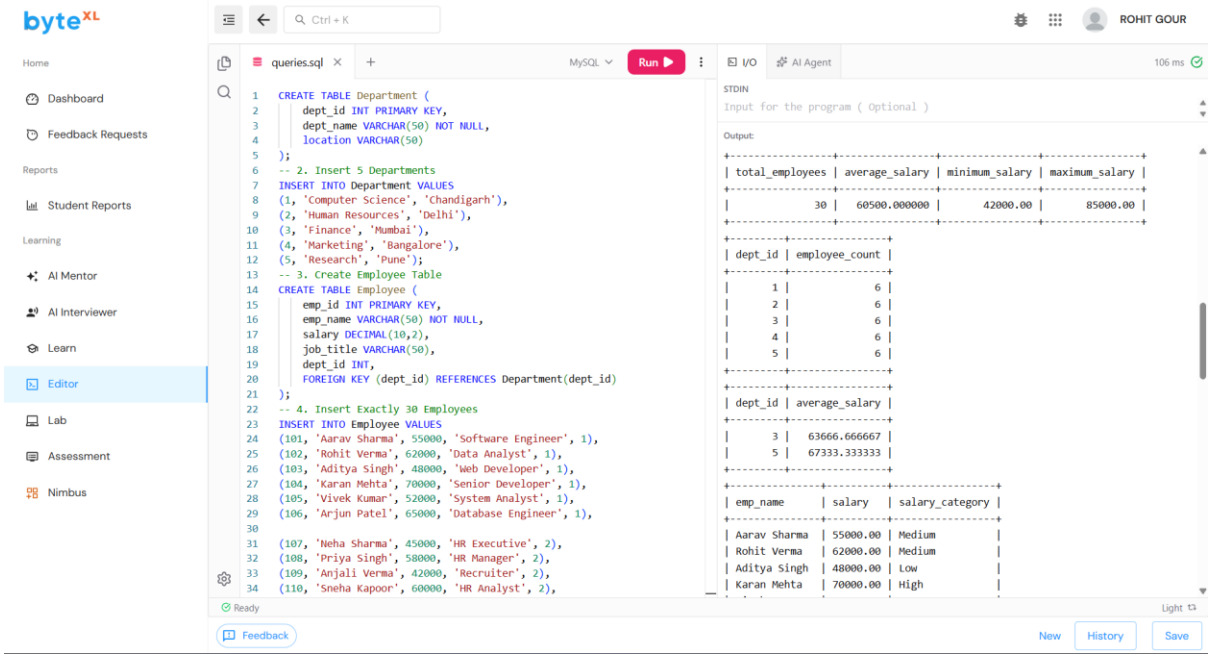
```
SELECT emp_name, salary
FROM Employee
ORDER BY salary DESC;
```

8. Project-wise Employee Count

```
SELECT project_id, COUNT(*) AS employee_count
FROM Employee_Project
GROUP BY project_id;
```

Result / Output





EXPERIMENT : 4

Aim

To execute SQL queries using INNER JOIN, LEFT JOIN, self-join, three-way join, correlated subqueries, EXISTS, simulated INTERSECT, and simulated EXCEPT, and to compare query execution plans using EXPLAIN.

Theory

Joins combine related records from multiple tables. An INNER JOIN returns matching records, while a LEFT JOIN also retains unmatched records from the left table. A self-join joins a table with itself. Subqueries allow one query to use the result of another query. EXISTS checks whether a matching record exists. MySQL does not directly provide INTERSECT and EXCEPT in traditional versions, so UNION and NOT EXISTS can be used to simulate their behavior. EXPLAIN displays the execution plan used by MySQL.

Source Code

The tables created in Experiment 3 are used for this experiment.

SQL Queries

1. INNER JOIN

```
SELECT e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

2. LEFT JOIN

```
SELECT d.dept_name, e.emp_name
FROM Department d
LEFT JOIN Employee e
ON d.dept_id = e.dept_id;
```

3. Self-Join

Display employees working in the same department.

```
SELECT e1.emp_name AS employee1,
    e2.emp_name AS employee2,
```

```
    e1.dept_id
FROM Employee e1
JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id;
```

4. Three-Way JOIN

```
SELECT e.emp_name, d.dept_name, ep.project_id
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id
JOIN Employee_Project ep
ON e.emp_id = ep.emp_id;
```

5. Correlated Subquery

Display employees earning more than the average salary of their own department.

```
SELECT emp_name, salary, dept_id
FROM Employee e
WHERE salary > (
  SELECT AVG(salary)
  FROM Employee
  WHERE dept_id = e.dept_id
);
```

6. EXISTS

Display departments having at least one employee.

```
SELECT dept_name
FROM Department d
WHERE EXISTS (
  SELECT 1
  FROM Employee e
```

```
    WHERE e.dept_id = d.dept_id
);
```

7. Simulated INTERSECT

Display department IDs that contain both employees and projects through employee assignments.

```
SELECT DISTINCT dept_id
FROM Employee
WHERE dept_id IN (
    SELECT DISTINCT e.dept_id
    FROM Employee e
    JOIN Employee_Project ep
    ON e.emp_id = ep.emp_id
);
```

8. Simulated EXCEPT

Display departments that do not have employees earning more than ₹70,000.

```
SELECT dept_id
FROM Department
WHERE dept_id NOT IN (
    SELECT dept_id
    FROM Employee
    WHERE salary > 70000
);
```

9. EXPLAIN

```
EXPLAIN
SELECT *
FROM Employee
WHERE dept_id = 1;
```

10. EXPLAIN with JOIN

EXPLAIN

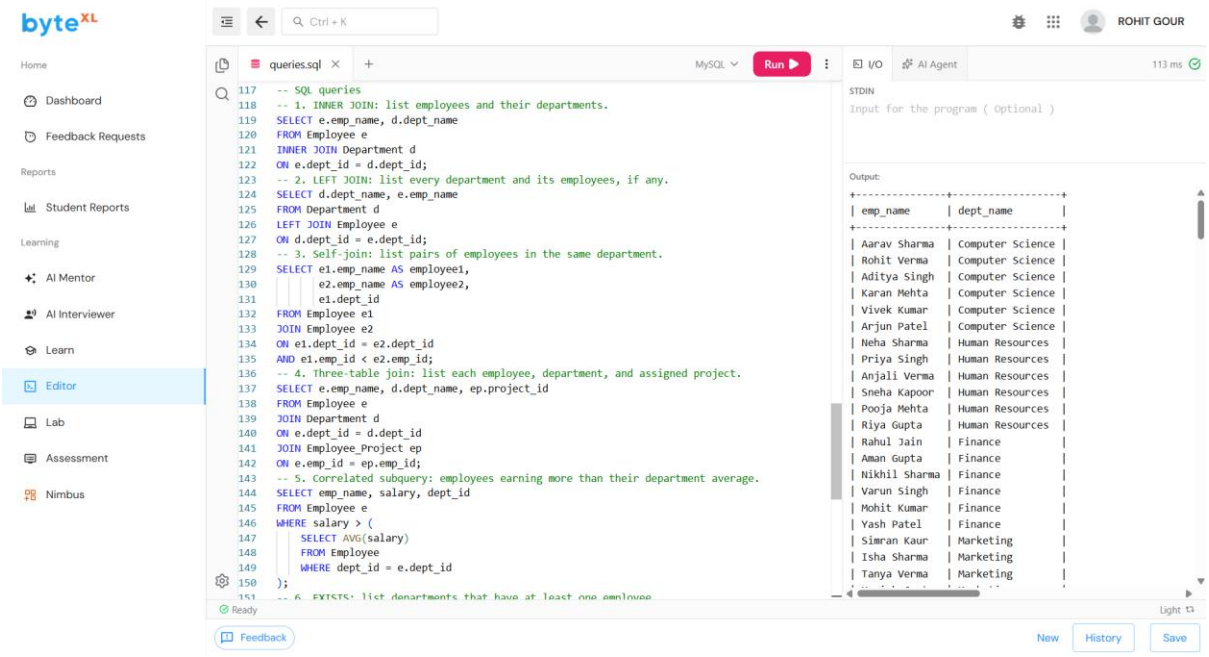
SELECT e.emp_name, d.dept_name

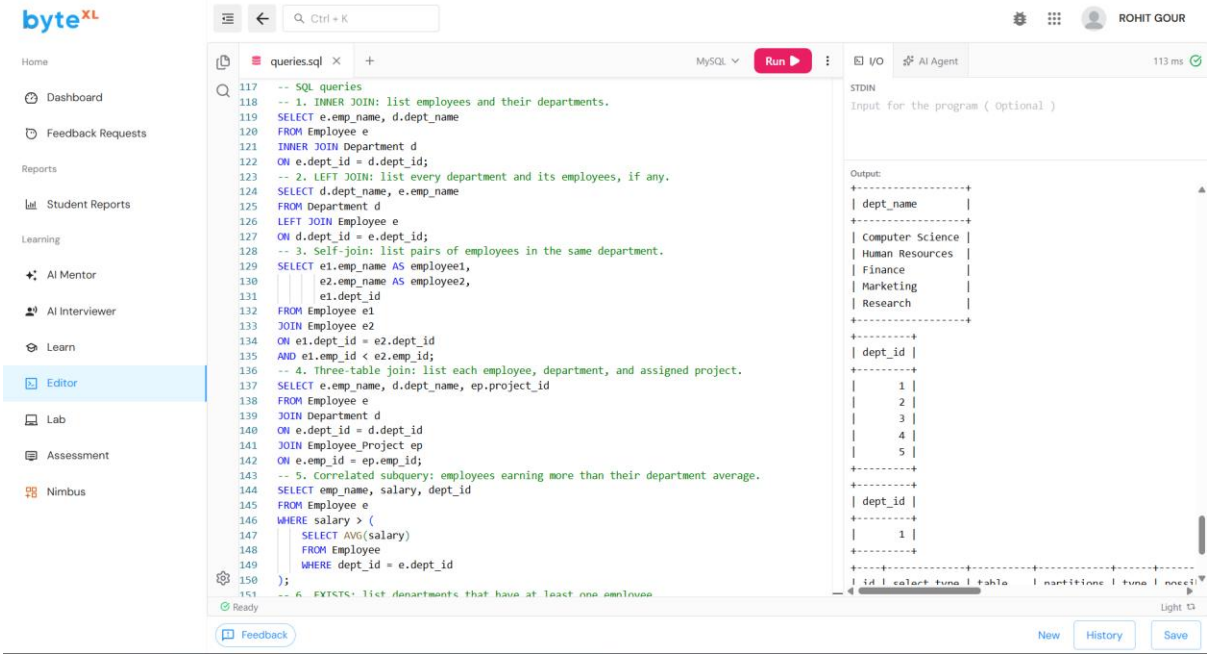
FROM Employee e

JOIN Department d

ON e.dept_id = d.dept_id;

Result / Output





EXPERIMENT : 5

Aim

To create SQL views for department salary summaries and employee hierarchy, test the updatability of views, and implement a recursive CTE to display employee reporting chains.

Theory

A view is a virtual table based on an SQL query. Views can simplify complex queries and provide controlled access to data. Views containing aggregate functions are generally not directly updatable. A recursive Common Table Expression (CTE) repeatedly executes a query to process hierarchical data such as employee-manager relationships.

Source Code

1. Add Manager Information

```
ALTER TABLE Employee

ADD manager_id INT;

UPDATE Employee

SET manager_id = NULL;


UPDATE Employee

SET manager_id = 104

WHERE emp_id IN (101, 102, 103);


UPDATE Employee

SET manager_id = 108

WHERE emp_id IN (107, 109, 110, 111);


UPDATE Employee

SET manager_id = 115

WHERE emp_id IN (113, 114, 116, 117);
```

```
UPDATE Employee
SET manager_id = 122
WHERE emp_id IN (119, 120, 121, 123);
UPDATE Employee
SET manager_id = 127
WHERE emp_id IN (125, 126, 128, 129);
```

2. Department Salary Summary View

```
CREATE VIEW department_salary_summary AS
SELECT dept_id,
    COUNT(*) AS employee_count,
    AVG(salary) AS average_salary,
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary
FROM Employee
GROUP BY dept_id;
```

3. Employee Hierarchy View

```
CREATE VIEW employee_hierarchy AS
SELECT e.emp_id,
    e.emp_name,
    e.manager_id,
    m.emp_name AS manager_name
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;
```

SQL Queries

1. Display Department Salary Summary

```
SELECT *
FROM department_salary_summary;
```

2. Display Employee Hierarchy

```
SELECT *
FROM employee_hierarchy;
```

3. Test View Updatability

The following simple view can be updated because it is based on a single table.

```
CREATE VIEW employee_basic AS
SELECT emp_id, emp_name, salary
FROM Employee;
UPDATE employee_basic
SET salary = 56000
WHERE emp_id = 101;
SELECT *
FROM employee_basic
WHERE emp_id = 101;
```

The aggregate salary-summary view is not directly updatable because it contains grouping and aggregate functions.

```
UPDATE department_salary_summary
SET average_salary = 60000
WHERE dept_id = 1;
```

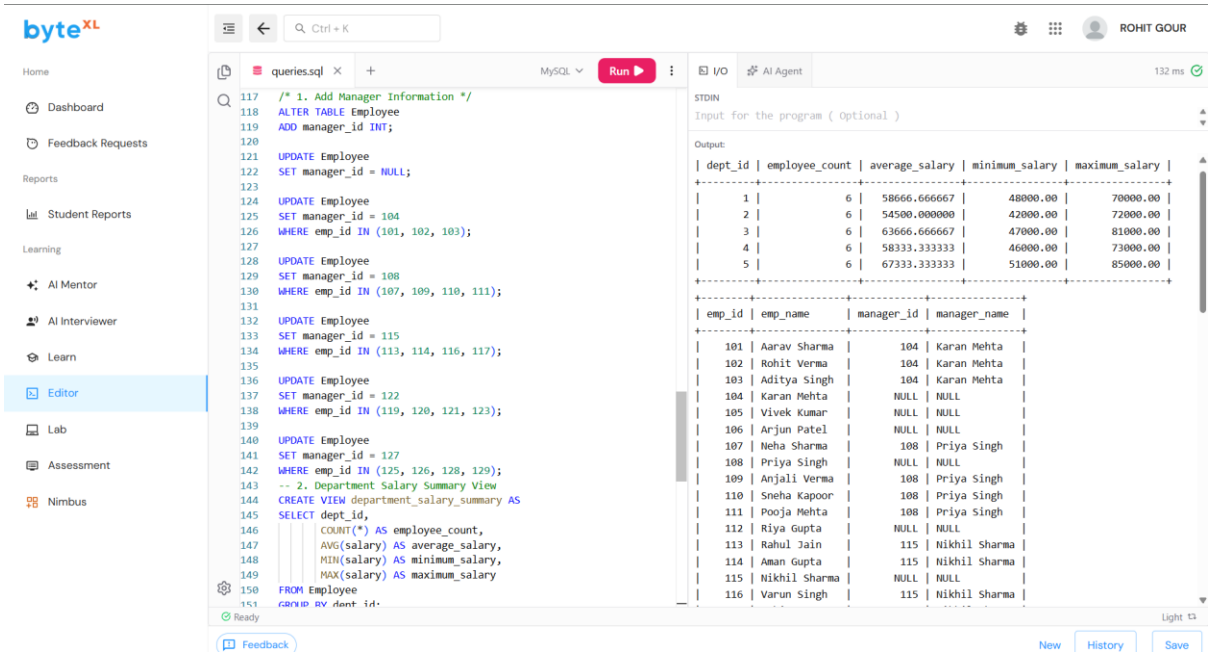
4. Recursive CTE for Reporting Chain

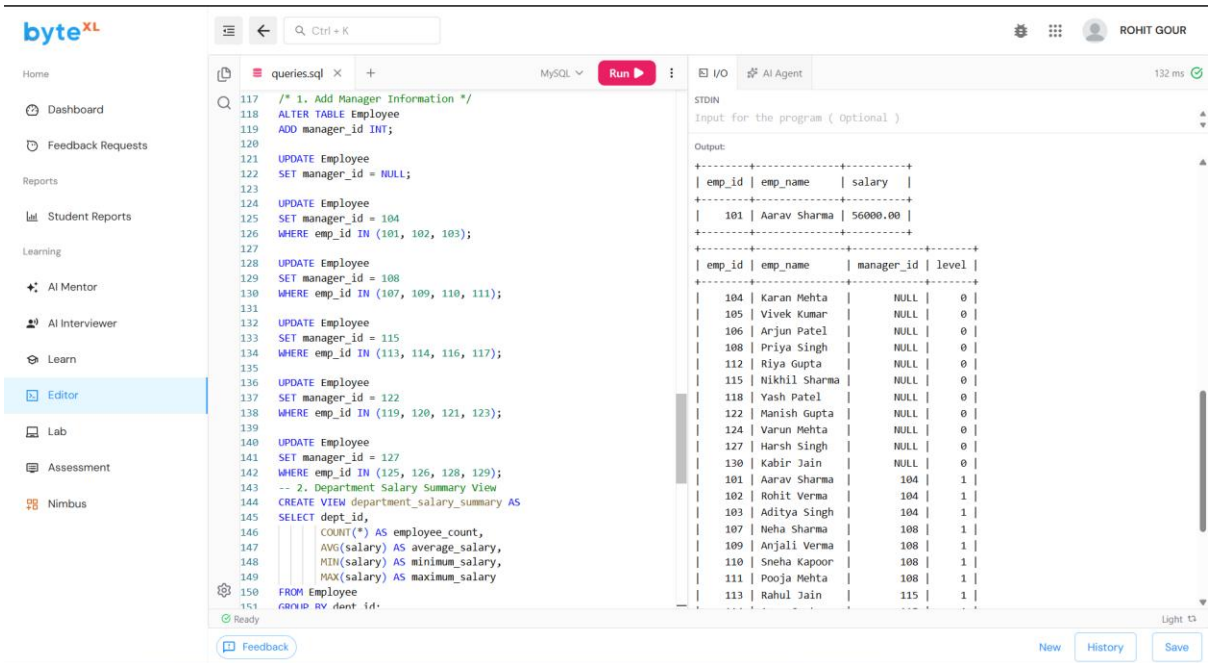
```
WITH RECURSIVE reporting_chain AS (
   SELECT emp_id, emp_name, manager_id, 0 AS level
   FROM Employee
   WHERE manager_id IS NULL
   UNION ALL

   SELECT e.emp_id, e.emp_name, e.manager_id, r.level + 1
   FROM Employee e
```

```
    JOIN reporting_chain r
    ON e.manager_id = r.emp_id
)
SELECT *
FROM reporting_chain
ORDER BY level, emp_id;
```

Result / Output





EXPERIMENT : 6

Aim

To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, implement triggers for salary validation and audit logging, and test different edge cases.

Theory

A stored procedure is a stored collection of SQL statements that can be executed when required. Procedures can contain parameters, conditions, transactions, and error handlers. Triggers execute automatically when specified database events occur. Validation triggers maintain data integrity, while audit triggers record important changes.

Source Code

1. Create Salary Audit Table

```
CREATE TABLE salary_audit (
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT,
   old_salary DECIMAL(10,2),
   new_salary DECIMAL(10,2),
   changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. Salary Validation Trigger

```
DELIMITER //

CREATE TRIGGER validate_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
   IF NEW.salary <= 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Salary must be greater than zero';
```

```
    END IF;
END//

DELIMITER ;
```

3. Salary Audit Trigger

```
DELIMITER //
CREATE TRIGGER salary_audit_trigger
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit
        (emp_id, old_salary, new_salary)
        VALUES
        (OLD.emp_id, OLD.salary, NEW.salary);
    END IF;
END//
DELIMITER ;
```

4. Stored Procedure

```
DELIMITER //
CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE emp_exists INT DEFAULT 0;
    DECLARE dept_exists INT DEFAULT 0;
```

```
    SELECT COUNT(*)
    INTO emp_exists
    FROM Employee
    WHERE emp_id = p_emp_id;

    SELECT COUNT(*)
    INTO dept_exists
    FROM Department
    WHERE dept_id = p_new_dept_id;

    IF emp_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    ELSEIF dept_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    ELSE
        UPDATE Employee
        SET dept_id = p_new_dept_id
        WHERE emp_id = p_emp_id;
    END IF;
END//
DELIMITER ;
```

SQL Queries

1. Transfer Employee

```
CALL transfer_employee(101, 2);
```

2. Test Invalid Employee

```
CALL transfer_employee(999, 2);
```

3. Test Invalid Department

CALL transfer_employee(101, 99);

4. Test Salary Validation

INSERT INTO Employee

VALUES (131, 'Test Employee', -5000, 'Tester', 1, NULL);

5. Test Salary Audit

UPDATE Employee

SET salary = 60000

WHERE emp_id = 102;

SELECT *

FROM salary_audit;

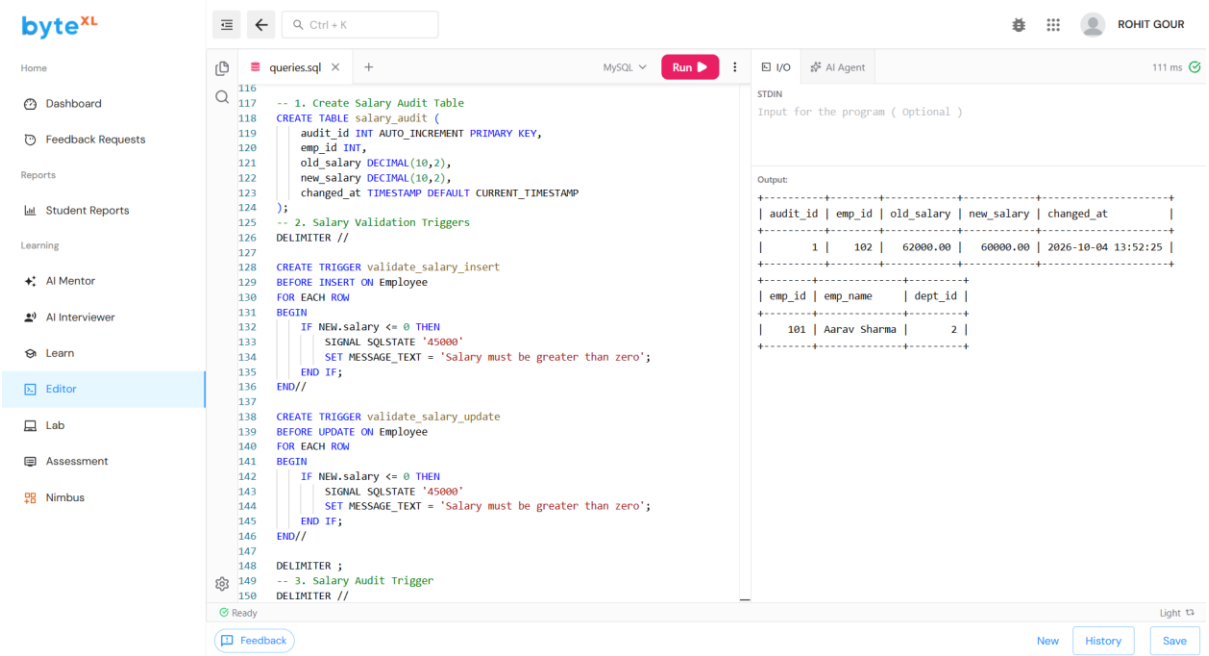
6. Verify Employee Transfer

SELECT emp_id, emp_name, dept_id

FROM Employee

WHERE emp_id = 101;

Result / Output



EXPERIMENT : 7

Aim

To identify functional dependencies in a denormalized Sales table and normalize the relation step-by-step from UNF to 3NF and BCNF. The decomposition is also examined for lossless join and dependency preservation using SQL queries.

Theory

Normalization is the process of organizing relational data to reduce redundancy and update anomalies.

UNF: Data may contain repeating groups or multiple values in a single field.

1NF: Each attribute contains atomic values and repeating groups are removed.

2NF: The relation is in 1NF and every non-key attribute is fully dependent on the complete candidate key.

3NF: The relation is in 2NF and contains no transitive dependency of non-key attributes on a key.

BCNF: For every functional dependency X → Y, X must be a superkey.

Source Code

1. Create UNF Sales Table

```
CREATE TABLE Sales_UNF (
    order_id INT,
    order_date DATE,
    customer_id INT,
    customer_name VARCHAR(50),
    customer_city VARCHAR(50),
    product_id INT,
    product_name VARCHAR(50),
    quantity INT,
    price DECIMAL(10,2),
    salesperson_id INT,
    salesperson_name VARCHAR(50)
```

);

2. Insert Sample Data

INSERT INTO Sales_UNF VALUES

(1, '2026-09-01', 101, 'Rohit', 'Delhi', 201, 'Laptop', 1, 60000, 501, 'Amit'),

(1, '2026-09-01', 101, 'Rohit', 'Delhi', 202, 'Mouse', 2, 800, 501, 'Amit'),

(2, '2026-09-02', 102, 'Neha', 'Mumbai', 201, 'Laptop', 1, 60000, 502, 'Priya'),

(3, '2026-09-03', 103, 'Aman', 'Pune', 203, 'Keyboard', 1, 1500, 501, 'Amit');

3. Functional Dependencies

The important functional dependencies are:

customer_id → customer_name, customer_city

product_id → product_name, price

salesperson_id → salesperson_name

order_id → order_date, customer_id, salesperson_id

(order_id, product_id) → quantity
The composite key of the sales relation is:
(order_id, product_id)

4. Convert UNF to 1NF

The repeating product information is represented as separate rows, giving each row atomic values.

CREATE TABLE Sales_1NF (

order_id INT,

order_date DATE,

customer_id INT,

customer_name VARCHAR(50),

customer_city VARCHAR(50),

product_id INT,

product_name VARCHAR(50),

quantity INT,

price DECIMAL(10,2),

```
    salesperson_id INT,
    salesperson_name VARCHAR(50),
    PRIMARY KEY (order_id, product_id)
);
```

5. Convert 1NF to 2NF

Separate attributes that depend only on part of the composite key.

```
CREATE TABLE Orders_2NF (
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT,
    customer_name VARCHAR(50),
    customer_city VARCHAR(50),
    salesperson_id INT,
    salesperson_name VARCHAR(50)
);

CREATE TABLE Products_2NF (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    price DECIMAL(10,2)
);

CREATE TABLE Order_Details_2NF (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id)
);
```

6. Convert 2NF to 3NF

Remove transitive dependencies.

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    customer_city VARCHAR(50)
);

CREATE TABLE Salespersons (
    salesperson_id INT PRIMARY KEY,
    salesperson_name VARCHAR(50)
);

CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT,
    salesperson_id INT,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
    FOREIGN KEY (salesperson_id) REFERENCES Salespersons(salesperson_id)
);

CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    price DECIMAL(10,2)
);
```

```
CREATE TABLE Order_Details (
    order_id INT,
    product_id INT,
    quantity INT,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

7. BCNF Verification

The determinants in the final relations are candidate keys:
customer_id → customer_name, customer_city

product_id → product_name, price

salesperson_id → salesperson_name

order_id → order_date, customer_id, salesperson_id

(order_id, product_id) → quantity
Therefore, the decomposed relations satisfy BCNF for these dependencies.

SQL Queries

1. Test Customer Dependency

```
SELECT customer_id, COUNT(DISTINCT customer_name) AS names
FROM Customers
GROUP BY customer_id;
```

2. Test Product Dependency

```
SELECT product_id, COUNT(DISTINCT price) AS prices
FROM Products
GROUP BY product_id;
```

3. Lossless Join Verification

```
SELECT o.order_id, c.customer_name, p.product_name, od.quantity
FROM Orders o
```

JOIN Customers c

ON o.customer_id = c.customer_id

JOIN Order_Details od

ON o.order_id = od.order_id

JOIN Products p

ON od.product_id = p.product_id;

4. Dependency Preservation Check

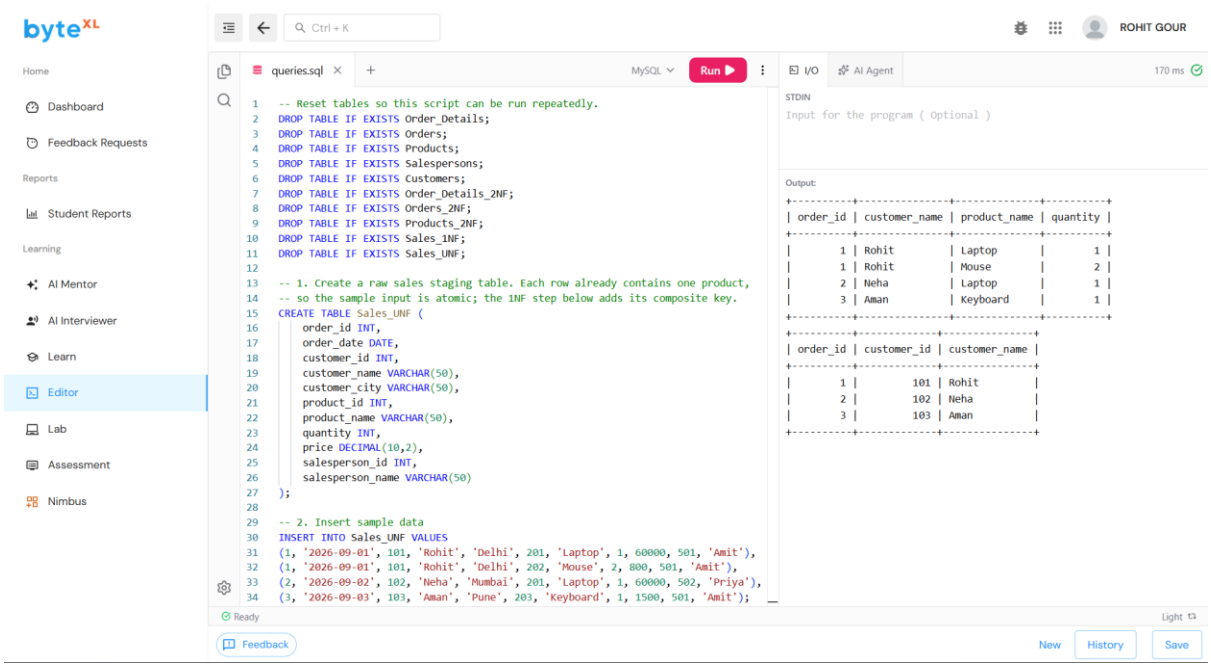
SELECT o.order_id, o.customer_id, c.customer_name

FROM Orders o

JOIN Customers c

ON o.customer_id = c.customer_id;

Result / Output



EXPERIMENT : 8

Aim

To implement banking-style transactions using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT, simulate partial failures, and demonstrate isolation levels and concurrency anomalies using two database sessions.

Theory

A transaction is a sequence of operations treated as a single logical unit. COMMIT permanently saves changes, while ROLLBACK cancels uncommitted changes. SAVEPOINT creates a point within a transaction to which a partial rollback can be performed. Transaction isolation controls how one transaction can interact with another concurrent transaction.

Source Code

1. Create Bank Account Table

```
CREATE TABLE Bank_Account (
   account_id INT PRIMARY KEY,
   account_holder VARCHAR(50),
   balance DECIMAL(10,2)
);
```

2. Insert Accounts

```
INSERT INTO Bank_Account VALUES
(1, 'Rohit', 50000),
(2, 'Aman', 30000),
(3, 'Neha', 40000);
```

SQL Queries

1. BEGIN and COMMIT

```
START TRANSACTION;

UPDATE Bank_Account
SET balance = balance - 5000
WHERE account_id = 1;
```

```
UPDATE Bank_Account
SET balance = balance + 5000
WHERE account_id = 2;

COMMIT;
```

2. ROLLBACK

```
START TRANSACTION;

UPDATE Bank_Account
SET balance = balance - 10000
WHERE account_id = 1;

ROLLBACK;
```

3. SAVEPOINT

```
START TRANSACTION;

UPDATE Bank_Account
SET balance = balance - 5000
WHERE account_id = 1;

SAVEPOINT transfer_point;

UPDATE Bank_Account
SET balance = balance + 5000
WHERE account_id = 2;

ROLLBACK TO transfer_point;
```

COMMIT;

4. Simulate Partial Failure

START TRANSACTION;

UPDATE Bank_Account

SET balance = balance - 5000

WHERE account_id = 1;

SAVEPOINT before_second_update;

UPDATE Bank_Account

SET balance = balance + 5000

WHERE account_id = 2;

ROLLBACK TO before_second_update;

COMMIT;

5. Set Isolation Level

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

6. Check Isolation Level

SELECT @@transaction_isolation;

Two-Session Concurrency Test

Open two MySQL Workbench query tabs/connections.

Session 1

START TRANSACTION;

UPDATE Bank_Account

SET balance = balance - 5000

WHERE account_id = 1;

SELECT *

FROM Bank_Account

WHERE account_id = 1;

Do not commit yet.

Session 2

START TRANSACTION;

SELECT *

FROM Bank_Account

WHERE account_id = 1;

COMMIT;

Session 1

COMMIT;

Non-Repeatable Read Demonstration

Set both sessions to READ COMMITTED.

Session 1:

SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT balance

FROM Bank_Account

WHERE account_id = 1;

Session 2:

START TRANSACTION;

UPDATE Bank_Account

SET balance = balance + 5000

WHERE account_id = 1;

COMMIT;

Session 1:

SELECT balance

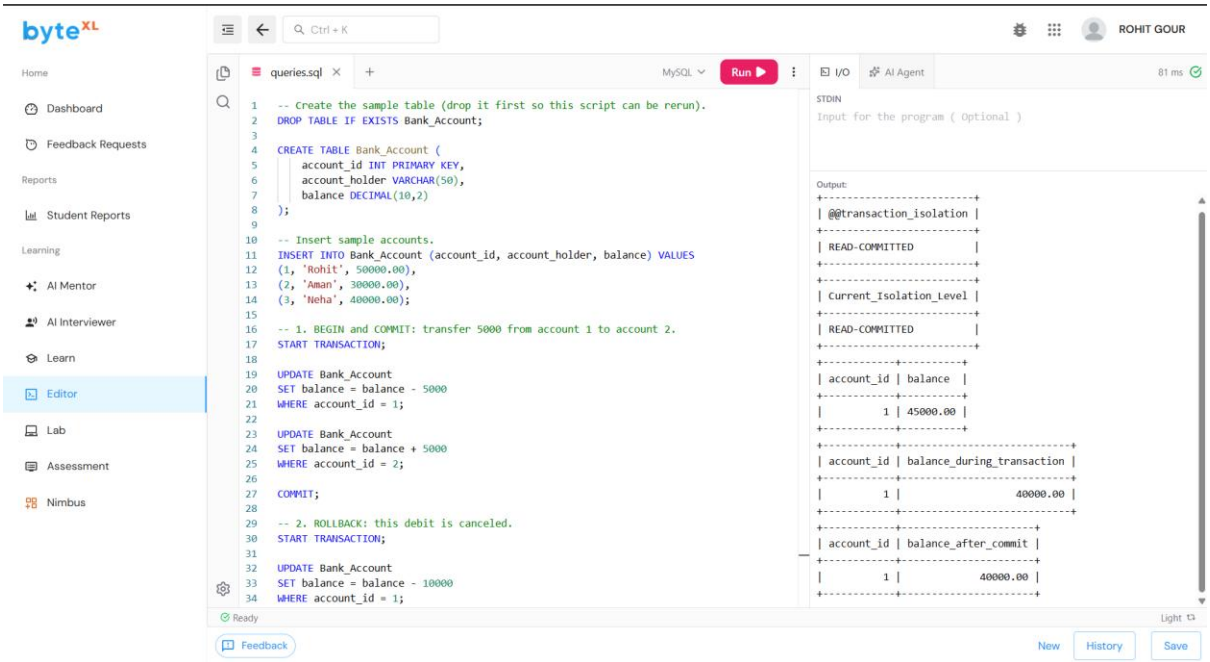
FROM Bank_Account

WHERE account_id = 1;

COMMIT;

The second read can return a different value because another transaction committed a change.

Result / Output



EXPERIMENT : 9

Aim

To create a Railway Booking table containing exactly 5,000 records, analyze query performance before and after creating single and composite indexes, and compare execution plans using EXPLAIN.

Theory

An index is a database structure that improves the speed of data retrieval. A single-column index is created on one attribute, while a composite index contains multiple attributes. Indexes can improve search performance but require additional storage and may increase the cost of insert and update operations. EXPLAIN is used to examine how MySQL executes a query.

Source Code

1. Create Railway Booking Table

```
CREATE TABLE Railway_Booking (
   booking_id INT PRIMARY KEY,
   passenger_name VARCHAR(50),
   train_no INT,
   source_station VARCHAR(50),
   destination_station VARCHAR(50),
   travel_date DATE,
   seat_no INT,
   fare DECIMAL(10,2)
);
```

2. Insert Exactly 5,000 Records

```
INSERT INTO Railway_Booking
(booking_id, passenger_name, train_no, source_station,
 destination_station, travel_date, seat_no, fare)
SELECT
   n,
```

```
  CONCAT('Passenger_', n),
  100 + MOD(n, 20),
  CASE MOD(n, 5)
    WHEN 0 THEN 'Delhi'
    WHEN 1 THEN 'Mumbai'
    WHEN 2 THEN 'Pune'
    WHEN 3 THEN 'Chandigarh'
    ELSE 'Bhopal'
  END,
  CASE MOD(n, 5)
    WHEN 0 THEN 'Mumbai'
    WHEN 1 THEN 'Pune'
    WHEN 2 THEN 'Delhi'
    WHEN 3 THEN 'Bhopal'
    ELSE 'Chandigarh'
  END,
  DATE_ADD('2026-01-01', INTERVAL MOD(n, 180) DAY),
  MOD(n, 100) + 1,
  300 + MOD(n, 1000)
FROM (
  SELECT a.n + b.n * 10 + c.n * 100 + d.n * 1000 + 1 AS n
  FROM
    (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
     SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
     SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) a,
    (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
     SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
     SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) b,
```

```
    (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
     SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
     SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) c,
    (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
     SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
     SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) d
) numbers
WHERE n <= 5000;
```

SQL Queries

1. Query Before Indexing

```
SELECT *
FROM Railway_Booking
WHERE train_no = 105;
```

2. Execution Plan Before Indexing

```
EXPLAIN
SELECT *
FROM Railway_Booking
WHERE train_no = 105;
```

3. Create Single Index

```
CREATE INDEX idx_train
ON Railway_Booking(train_no);
```

4. Query After Single Index

```
SELECT *
FROM Railway_Booking
WHERE train_no = 105;
```

5. Execution Plan After Single Index

```
EXPLAIN
SELECT *
```

```
FROM Railway_Booking

WHERE train_no = 105;
```

6. Query for Composite Index

```
SELECT *

FROM Railway_Booking

WHERE train_no = 105

AND travel_date = '2026-03-01';
```

7. Create Composite Index

```
CREATE INDEX idx_train_date

ON Railway_Booking(train_no, travel_date);
```

8. Execution Plan After Composite Index

```
EXPLAIN

SELECT *

FROM Railway_Booking

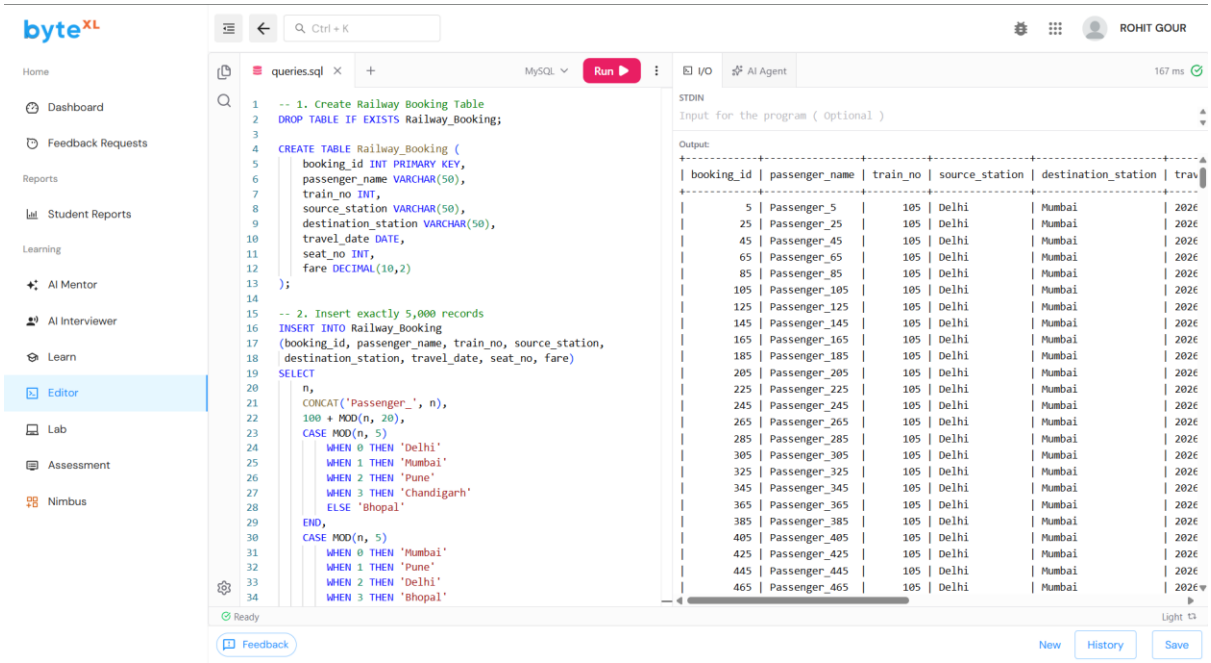
WHERE train_no = 105

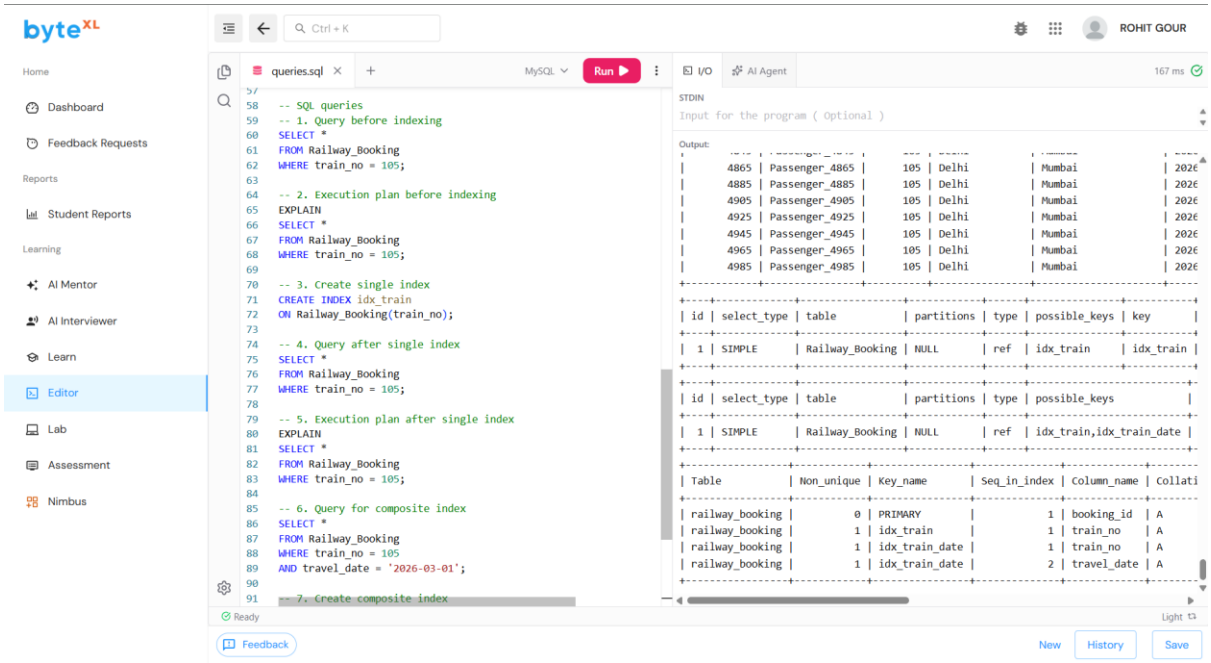
AND travel_date = '2026-03-01';
```

9. Compare Indexes

```
SHOW INDEX FROM Railway_Booking;
```

Result / Output





EXPERIMENT : 10

Aim

To generate a dataset containing exactly 10,000 rows, identify five slow queries, analyze their execution plans using EXPLAIN, optimize them using indexing and query rewriting, and compare query execution before and after optimization.

Theory

Query optimization improves database performance by reducing the amount of data scanned and selecting efficient execution methods. Indexing allows the database to locate records faster, while query rewriting can eliminate unnecessary operations. EXPLAIN helps identify table scans, indexes used, and other execution details.

Source Code

1. Create Dataset Table

```
CREATE TABLE Sales_Data (
   sale_id INT PRIMARY KEY,
   customer_id INT,
   product_id INT,
   region VARCHAR(30),
   quantity INT,
   amount DECIMAL(10,2),
   sale_date DATE
);
```

2. Generate Exactly 10,000 Rows

```
INSERT INTO Sales_Data
(sale_id, customer_id, product_id, region, quantity, amount, sale_date)
SELECT
   n,
   MOD(n, 1000) + 1,
   MOD(n, 100) + 1,
   CASE MOD(n, 4)
```

```
        WHEN 0 THEN 'North'
        WHEN 1 THEN 'South'
        WHEN 2 THEN 'East'
        ELSE 'West'
    END,
    MOD(n, 10) + 1,
    (MOD(n, 10) + 1) * 500,
    DATE_ADD('2026-01-01', INTERVAL MOD(n, 365) DAY)
FROM (
    SELECT a.n + b.n * 10 + c.n * 100 + d.n * 1000 + 1 AS n
    FROM
      (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
       SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
       SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) a,
      (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
       SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
       SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) b,
      (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
       SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
       SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) c,
      (SELECT 0 n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL
       SELECT 3 UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL
       SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL SELECT 9) d
) numbers
WHERE n <= 10000;
```

SQL Queries

Slow Query 1 – Customer Search

```
SELECT *
FROM Sales_Data
WHERE customer_id = 500;
```

Analyze Query 1

```
EXPLAIN
SELECT *
```

```
FROM Sales_Data
WHERE customer_id = 500;
```

Optimize Query 1

```
CREATE INDEX idx_customer
ON Sales_Data(customer_id);
EXPLAIN
SELECT *
FROM Sales_Data
WHERE customer_id = 500;
```

Slow Query 2 – Region and Date Search

```
SELECT *
FROM Sales_Data
WHERE region = 'North'
AND sale_date = '2026-03-01';
```

Analyze Query 2

```
EXPLAIN
SELECT *
FROM Sales_Data
WHERE region = 'North'
AND sale_date = '2026-03-01';
```

Optimize Query 2

```
CREATE INDEX idx_region_date
ON Sales_Data(region, sale_date);
EXPLAIN
SELECT *
FROM Sales_Data
WHERE region = 'North'
```

```
AND sale_date = '2026-03-01';
```

Slow Query 3 – Product Search

```
SELECT *
FROM Sales_Data
WHERE product_id = 50;
```

Analyze Query 3

```
EXPLAIN
SELECT *
FROM Sales_Data
WHERE product_id = 50;
```

Optimize Query 3

```
CREATE INDEX idx_product
ON Sales_Data(product_id);
EXPLAIN
SELECT *
FROM Sales_Data
WHERE product_id = 50;
```

Slow Query 4 – Date Range Search

```
SELECT *
FROM Sales_Data
WHERE sale_date BETWEEN '2026-01-01' AND '2026-06-30';
```

Analyze Query 4

```
EXPLAIN
SELECT *
FROM Sales_Data
WHERE sale_date BETWEEN '2026-01-01' AND '2026-06-30';
```

Optimize Query 4

```
CREATE INDEX idx_sale_date
ON Sales_Data(sale_date);
EXPLAIN
SELECT *
FROM Sales_Data
WHERE sale_date BETWEEN '2026-01-01' AND '2026-06-30';
```

Slow Query 5 – Query Rewriting

```
SELECT *
FROM Sales_Data
WHERE quantity + 1 > 5;
```

Analyze Query 5

```
EXPLAIN
SELECT *
FROM Sales_Data
WHERE quantity + 1 > 5;
```

Optimized Query

Rewrite the condition so that the column does not need to be calculated for every row.

```
SELECT *
FROM Sales_Data
WHERE quantity > 4;
```

Analyze Optimized Query

```
EXPLAIN
SELECT *
FROM Sales_Data
WHERE quantity > 4;
```

Execution Time Comparison

The queries can be executed before and after optimization and their execution times can be compared in MySQL Workbench. The EXPLAIN output can also be used to compare the access type, possible keys, selected key, and number of rows examined.

Result / Output

